PROJECT NEXUS 2026 · AI INNOVATION CHALLENGE

Full-Stack App Deployment

with AI APIs & Cloud

Build a web application from an empty folder, connect an AI API, and put it on the internet.

No prior experience of web development, servers, or deployment is assumed. Nothing in this guide costs money.

Workshop 4 of 10 · 31 August 2026

IEEE Multimedia University Student Branch
Co-organisers: E3S2 UTP, IEEE PES MMU SBC, EWB MMU, IEM MMU

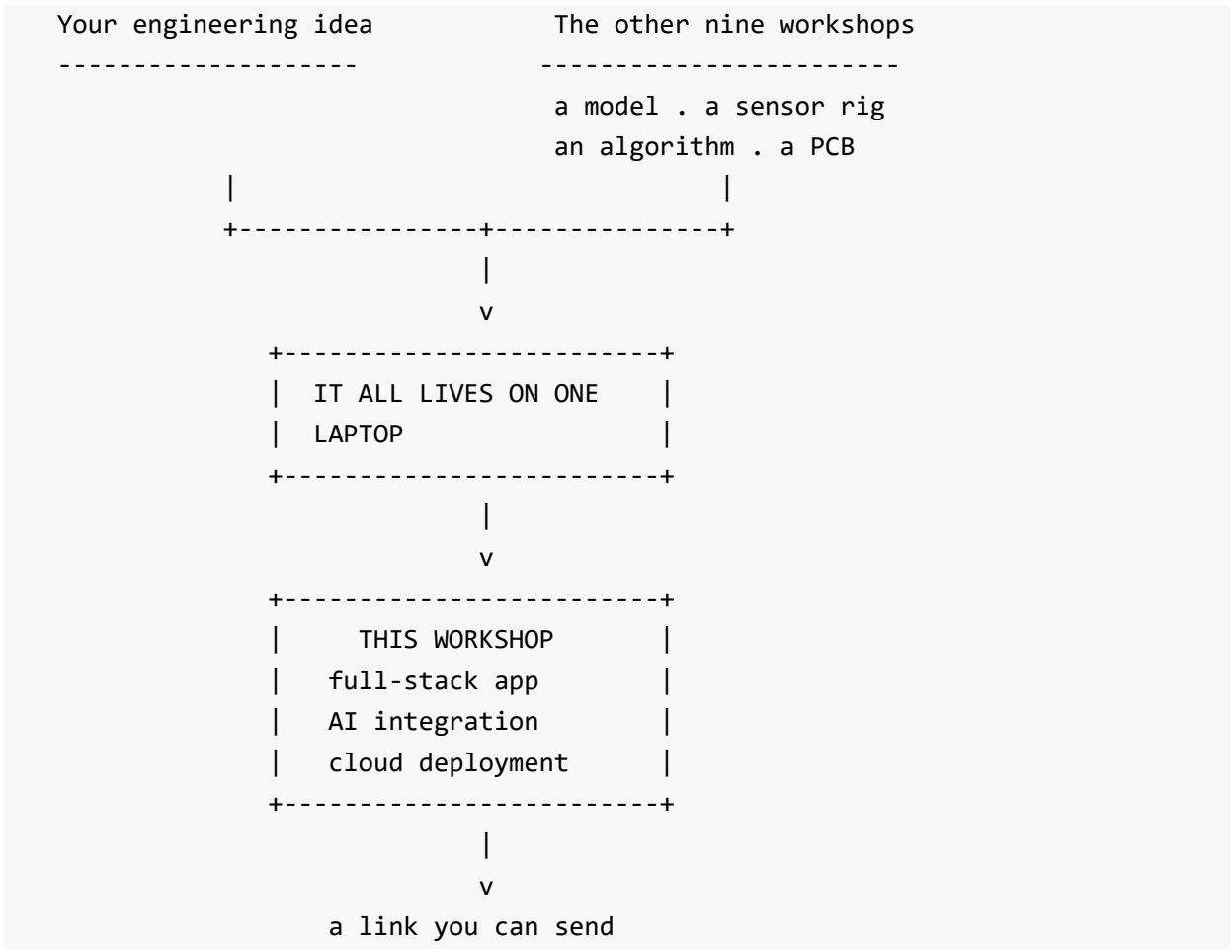
Every command and every line of code in this guide was executed and verified before publication.

Contents

Why this workshop exists

Nine of the ten workshops in this series teach you to build something: a model, a sensor rig, an algorithm, a circuit board.

This is the one where somebody else gets to see it.



A trained model is invisible to anyone not sitting at the laptop that trained it. A sensor reading in a terminal cannot be shown to a judge, a supervisor, or a collaborator in another city.

This workshop is the bridge across that gap. It is not about AI, and it is not about hardware. It is about the step that turns either one into something with an address.

How to use this guide

This guide takes you from a computer with nothing installed to a working web application running at a public address that anyone can open.

You do not need any prior experience of web development, servers, or deployment. You do need to be able to program — if you have written Python, C for a microcontroller, or anything similar, you have enough.

What you will build

A **sensor triage service**. Something sends a reading; your code decides whether it is safe; an AI writes a short explanation; the result is stored and displayed on a web page.

That shape fits every Project Nexus track, because every track measures a number that should stay inside a range — a body temperature, a CO2 level, a soil moisture percentage, a motor temperature.

What it costs

Nothing. No API key, no credit card, no paid account. The application runs fully in a mode that needs no AI service, and the hosting we use has a free tier that does not ask for a card.

If you do have an AI key, a section shows you how to use it, and what it costs. (Roughly one tenth of a cent per request.)

The six parts

Part	What it covers	Code?
0	Setting up your computer, from nothing	no
1	What these technologies actually are	no
2	**The fast path** — a model out of a notebook and online in 20 minutes	yes
3	Building the full application, in five stages	yes
4	Putting it on the internet	yes
5	Where to go next	yes

Parts 0 and 1 contain no code. If you already have Python, VS Code and Git working, and you know what a server and an API are, start at Part 2.

There are two paths, and they are both real. Part 2 gets a trained model online in about twenty minutes with no HTML and no server code. Part 3 builds a full application that a device can send readings to. Which you need depends on your project, and Part 2 ends with a table that decides it.

The companion repository

https://github.com/Shamsu3100/demon-

The fast path lives in `fastpath/`, and every stage of Part 3 is a complete, working folder:

```
fastpath/          the notebook, the model, and the Gradio app
stages/stage1      a server that answers
stages/stage2      the web page appears
stages/stage3      your code decides
stages/stage4      the readings are stored
stages/stage5      the AI explains
stages/stage6      the deployment files added
```

If you fall behind, copy the stage you need and carry on. You do not have to start again.

Every line of code in this guide was run and tested before it was printed here.

How to read the boxes

> Boxes like this one explain a term, warn about a trap, or give a piece of context. They are not optional extras — several of them describe mistakes that cost a full day if you meet them unprepared.

Part 0 — Setting up your computer

Nothing in this part is about our project. It installs the four tools every developer uses, and creates the two free accounts we need at the end.

If you already have Python, VS Code and Git working, skip to the checklist at the end and confirm all four commands run.

Allow about 45 minutes the first time. You only ever do this once.

> Instructions are written for **Windows**, because most laptops in the room are Windows. Where macOS or Linux differs, it is marked like this: **macOS / Linux:** …

0.1 The terminal

What it is

A window where you type commands instead of clicking. Every tool we use today is driven from it.

It feels unfamiliar for about twenty minutes, then it stops being frightening. You are not going to break anything: the commands in this guide only create files and start programs.

Opening it

Windows — any of these work:

1. Press the **Windows key**, type `powershell`, press **Enter**.
2. Or: right-click the **Start** button -> **Terminal** or **Windows PowerShell**.
3. Or, best while working: open your project folder in File Explorer, click the address bar, type `powershell`, press **Enter**. This opens a terminal already in that folder.

macOS: press `Cmd + Space`, type `terminal`, press **Enter**.

You will see a line ending in > (Windows) or $ (macOS). That is the prompt. It is waiting for you.

The five commands you actually need

Command	What it does
`cd foldername`	go **into** a folder
`cd ..`	go **back up** one folder
`dir`	list what is in this folder (**macOS / Linux:** `ls`)
`cls`	clear the screen (**macOS / Linux:** `clear`)
`Ctrl + C`	stop whatever is running

Two habits worth building now:

- **Press `Tab` to complete a name.** Type `cd sen` then `Tab` and Windows finishes `sensor-triage` for you. Fewer typos, much faster.
- **Look at the prompt before you run anything.** It shows which folder you are in. Most "it doesn't work" moments are really "I am in the wrong folder".

Two Windows traps

Paths with spaces need quotes.

```
cd "E:\Nexus Training\workshop"
```

Without the quotes, Windows reads `E:\Nexus` and `Training\workshop` as two separate things and fails.

`cd` will not change drive letters in Command Prompt. If your prompt says `C:\>` and your work is on `E:`, type the drive letter on its own first:

```
E:
cd "E:\Nexus Training\workshop"
```

PowerShell does not have this problem. Command Prompt does. If a `cd` seems to do nothing at all, this is usually why.

0.2 Show file extensions (Windows — do this first)

Windows hides the end of filenames by default. That single setting causes a problem later in this workshop, so fix it now.

Later you will create a file called exactly `.env`. With extensions hidden, Notepad silently saves it as `.env.txt`, your program cannot find it, and nothing on screen tells you why.

1. Open **File Explorer**
2. Click the **View** menu
3. Turn on **File name extensions**

(Windows 10: View tab -> tick "File name extensions". Windows 11: View -> Show -> File name extensions.)

You should now see `main.py` rather than `main`. Two clicks, and it saves an hour of confusion.

0.3 Python

What it is

The language we write in. Your computer probably does not have it, and the one that ships with macOS is too old to use.

Installing it

1. Go to **https://www.python.org/downloads/**
2. Click the big yellow **Download Python 3.x** button
3. Run the file you downloaded

On the first installer screen, before clicking anything:

> **[x] Tick "Add python.exe to PATH"**
>
> It is a small checkbox near the bottom and it is **off** by default.
> If you miss it, your terminal will say `python is not recognized` and you will have to reinstall.
> This is the single most common setup failure.

Then click **Install Now** and wait.

macOS: the installer has no PATH checkbox; it handles this itself.

Linux: `sudo apt install python3 python3-venv python3-pip`

Checking it worked

Close your terminal and open a new one — this matters, because an already open terminal does not know about newly installed programs.

```
python --version
```

You should see something like:

```
Python 3.13.7
```

Any version **3.10 or higher** is fine.

If it says `python is not recognized`: the PATH checkbox was missed. Run the installer again, choose **Modify**, and make sure PATH is ticked. Or try `py --version`, which sometimes works when `python` does not.

macOS / Linux: use `python3 --version`. On those systems, plain `python` often means an old version 2, which will not work.

0.4 VS Code

What it is

The editor we write code in. You could use Notepad, but VS Code colours your code, points out mistakes as you type, and has a terminal built in.

It is free, from Microsoft, and it is what most professionals use.

Installing it

1. Go to **https://code.visualstudio.com**
2. Click **Download**, run the installer, accept the defaults
3. On the "Select Additional Tasks" screen, tick **"Add 'Open with Code' action to directory context menu"** — it lets you right-click any folder and open it straight in the editor

Two things to know

Always open a *folder*, not a file. `File -> Open Folder`. VS Code shows everything in that folder down the left side, and its built-in terminal starts in the right place. Opening a single file gives you none of that.

The built-in terminal: `View -> Terminal`, or `Ctrl + ~` (the key above Tab). This is where you will type most commands, and it already knows which folder you are in.

The Python extension

The first time you open a `.py` file, VS Code offers to install the Python extension. **Accept.** It adds error highlighting and autocomplete.

0.5 Git

What it is — before you install it

Git records versions of your code. Every time you save a checkpoint (a **commit**), it remembers exactly what every file looked like at that moment.

Two reasons we need it:

1. **You can go back.** If you break something on Thursday, you can return to Tuesday's working version.
2. **It is how code reaches a server.** Our hosting provider does not want a zip file. It collects your code from GitHub, and GitHub speaks git.

> **Git and GitHub are different things. Git** is the program on your computer that records versions. **GitHub** is a website that stores copies of those versions online. Git works perfectly well with no GitHub account. GitHub is where you put it so other computers can reach it.

Installing it

1. Go to **https://git-scm.com/downloads** and click **Windows**
2. Run the installer

The installer asks a lot of questions. **Accept every default.** They are sensible, and none of them matter for this workshop.

macOS: type `git --version` in Terminal — macOS offers to install it.

Linux: `sudo apt install git`

Checking it worked

Open a **new** terminal:

```
git --version
```

```
git version 2.54.0.windows.1
```

Any version is fine.

Tell git who you are (one time, ever)

Git labels every checkpoint with a name and an email. Set them once:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```

Use the same email you will use for GitHub in the next section.

0.6 A GitHub account

What it is

A website that stores your code online, for free. It is where almost all open source software lives, and it is how our hosting provider will collect our project.

Your account is also a public portfolio. Employers look at it.

Creating one

1. Go to **https://github.com/signup**
2. Enter your **email address** — use one you can open right now
3. Choose a **password**
4. Choose a **username**

> **Choose the username carefully.** It becomes part of every web address you create: `github.com/your-username/your-project`. It is public and hard to change later.
> Something close to your real name is a good choice. Avoid anything you would not want on a CV.

5. Solve the puzzle it shows you (this proves you are a person)
6. GitHub emails you an **8-digit code**. Open your email, copy it, paste it in
7. When it asks about a plan, choose the **free** option. Free is enough for everything in this workshop and far beyond

What a repository is

You will see the word “repository” (usually shortened to **repo**) everywhere.

A repository is just **one project’s folder, stored on GitHub, with all of its history**. One project, one repo.

We create ours in Part 4. Nothing to do now.

0.7 A Render account

What it is

Render is the company that will run our application on a computer that never turns off. Their free plan needs **no credit card**.

We cover what Render actually is, and what the alternatives are, in Part 1. Right now we are only creating the account.

Creating one

1. Go to **https://render.com**
2. Click **Get Started** (or **Sign Up**)
3. Choose **Sign in with GitHub**

Signing in with GitHub means you do not create another password, and Render can see the repositories you tell it to.

4. GitHub asks you to **Authorize Render**

> **What is that screen asking?**
> It is GitHub checking that you are happy for Render to read your code. This is normal and required — Render cannot deploy code it cannot read.
> Get into the habit of reading these screens rather than clicking through. Check the name of the app asking, and what access it wants. Here it should say Render, and it should be asking about repositories.

5. Choose the **free** plan when offered. No card is required

Nothing else to do here. We come back to it in Part 4.

0.8 One thing that will trip you up on Windows

Not now — in Part 3, when you activate a virtual environment. It is included here so you recognise it when it happens.

You will run this:

```
.venv\Scripts\activate
```

and PowerShell may refuse:

```
.venv\Scripts\activate : File ... cannot be loaded because
running scripts is disabled on this system.
```

Nothing is broken. Windows blocks scripts by default as a security measure.

The fix, run once in PowerShell:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Type `Y` and press Enter. This allows scripts you wrote yourself, and scripts that are signed. It applies only to your own user account, not the whole machine.

Then try `.venv\Scripts\activate` again.

0.9 Checklist

Open a **new** terminal and run all four. Do not continue until each one prints a version number.

```
python --version
```

```
pip --version
```

```
git --version
```

```
code --version
```

(If `code --version` fails, that is the least important one — VS Code still works, it just is not on your PATH. Everything else must work.)

And confirm:

- ☐ File name extensions are visible in File Explorer
- ☐ You can open a folder in VS Code and use its built-in terminal
- ☐ You are logged in at github.com
- ☐ You are logged in at render.com

If something will not work

What you see	What it means	Fix
`python is not recognized`	the PATH checkbox was missed	reinstall Python, tick **Add python.exe to PATH**
`git is not recognized`	terminal opened before install	close the terminal, open a new one

What you see	What it means	Fix
Command seems to do nothing	wrong drive (Command Prompt)	type `E:` on its own first
`The system cannot find the path`	a space in the path	wrap it in quotes: `cd "My Folder"`
`running scripts is disabled`	Windows script blocking	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
Your `.env` file is called `.env.txt`	extensions are hidden	turn on file extensions (section 0.2), rename it
macOS: `python: command not found`	macOS uses `python3`	use `python3` and `pip3` everywhere

Still stuck? Note the **exact** error text and ask. The message almost always names the problem, and reading it carefully is a skill worth building now.

Setup is done. **Part 1** explains what these technologies actually are, before we write any code.

Part 1 — What these technologies actually are

No code in this part.

By the end of it you should be able to read a job advert, a tutorial, or a hosting company's pricing page and know what the words mean — and choose where to put your own project, with a reason.

1.1 What "full-stack" means

An application people reach over the internet has three parts.

Part	What it is	Where it runs	Who can see it
Frontend	the page: buttons, text, layout	in the visitor's browser	**everyone**
Backend	your program: the logic	on a server	only you
Data	what is kept between visits	on a server	only you

"Full-stack" means you build all three. That is what we are doing today.

The line that matters most

Everything the frontend receives has been handed to the visitor to keep.

When someone opens your page, you are not showing it to them. You are sending them a copy. The HTML, the styling, the JavaScript, and anything you left inside it.

Prove it now: open any website, press `F12`, and look at the **Sources** or **Network** panel. Every file that page sent is listed, and you can read all of them.

There is no setting that prevents this. It is how browsers work.

That single fact decides where your API key is allowed to live — which is section 1.8, and the reason the backend exists at all.

1.2 Request and response

Every action on the web is two steps.

```
browser  ──────── request ────────>  server
browser  <──────── response ────────  server
```

- A **request** is a question sent over the internet.
- A **response** is the answer.

Opening a page is a request and a response. Pressing a button is a request and a response. Loading an image, submitting a form, sending a message — all the same shape, repeated very fast.

The two kinds you will use

	Meaning	Example
GET	*give me something*	opening a page
POST	*here is something*	submitting a form

There are others (`PUT`, `DELETE`, `PATCH`), and they follow the same idea. You need these two today.

What a status code is

Every response carries a three-digit number saying how it went. You will see these constantly:

Code	Meaning
200	fine
404	there is nothing at that address
422	you sent something, but it was the wrong shape
500	your code crashed

Learning to read these saves enormous time. A 404 and a 500 are completely different problems: 404 means your address is wrong, 500 means your address was right and the code behind it broke.

Your job as a backend developer is to write the code that decides what goes in the response. Not the internet, not the browser. Just that.

1.3 What a server is

A computer that stays switched on, waiting for requests.

That is the whole idea. Not a special kind of machine — an ordinary computer with a boring job and a permanent address.

For most of this workshop, **the server is your own laptop.** You will run one in Part 3 and visit it in your own browser. It is a real server; it is just not reachable by anyone else.

Deploying, in Part 4, means moving that same program onto a computer that never sleeps and does have a public address.

What a port is

You will type addresses like `127.0.0.1:8000`. The `:8000` is the **port**.

One computer can run many programs that all listen for requests. The port is how it knows which one you want — like flat numbers in an apartment building, where the address gets you to the building and the number gets you to the door.

- `127.0.0.1` (also written `localhost`) always means **this computer, right here**
- `:8000` means the program listening on port 8000

Two things to remember:

1. **Two programs cannot use the same port.** `Address already in use` means something else is already there — usually a server you forgot to stop.
2. **`localhost` is relative.** On your laptop it means your laptop. On a server it means the server. This causes a real and confusing bug, covered in Part 5.

1.4 What "the cloud" actually is

Someone else's computer, in a building somewhere, rented to you by the hour.

That is not a joke or a simplification. When you deploy to "the cloud", a company with a warehouse full of machines gives you a slice of one, keeps it powered and connected, and charges you for it. The word makes it sound weightless; it is a building with air conditioning and a large electricity bill.

What you are actually paying for

Uptime	someone else keeps the power and network on
An address	a permanent, reachable location
Maintenance	security patches, failed disks, cooling
Elasticity	more capacity when you need it, less when you do not

You could do all of this yourself with a computer at home. People do. You would be responsible for power cuts, your home internet address changing, security updates, and the machine dying at 3am.

For a student project, renting is obviously correct. Knowing *what* you are renting is the point of the next section.

1.5 The four ways to host code

This is the section that transfers to every project you ever build. Hosting comes in four shapes, and they differ in one thing: **how much you hand over.**

1. Static hosting

- **You give them:** finished files — HTML, CSS, images, JavaScript
- **They give you:** an address that serves those files, very fast
- **Examples:** GitHub Pages, Netlify, Cloudflare Pages, Vercel

Nothing runs on the server. It hands out files exactly as you uploaded them.

- Almost always free, and extremely fast
- **Cannot run Python.** Cannot use a database. Cannot hold a secret
- Anything it sends is public, by definition

Right for a portfolio, documentation, or a landing page. **Wrong for us**, because we need Python to run and a key to stay hidden.

2. Platform hosting <- *what we will use*

- **You give them:** your code, and a command to start it
- **They give you:** a machine that runs it, with an address and HTTPS
- **Examples:** Render, Railway, Heroku, Fly.io, PythonAnywhere, Google App Engine

You never touch the operating system. You do not install Python, configure a web server, or set up certificates. You push code; it runs.

- Fastest path from code to a working address
- Usually has a free tier
- Less control — you get their machine, their versions, their limits
- Free tiers sleep when idle, and wake slowly

Right for prototypes, small applications, and almost every student project.

3. Server hosting

- **You give them:** nothing. You rent an empty computer
- **They give you:** a bare Linux machine and its address
- **Examples:** AWS EC2, DigitalOcean, Linode, Hetzner, Azure Virtual Machines

You install Python. You configure a web server. You set up HTTPS certificates, a firewall, automatic restarts, and security updates. Forever.

- Complete control; run anything
- Cheapest at scale
- **You are now a system administrator**, whether or not you wanted to be
- No free tier that lasts; typically a few dollars a month

Right when you have outgrown a platform, or need something unusual installed. Wrong as a first deployment — you will spend the workshop on Linux configuration rather than on your application.

4. Serverless

- **You give them:** a single function
- **They give you:** that function run on demand, as often as needed
- **Examples:** AWS Lambda, Google Cloud Functions, Azure Functions, Cloudflare Workers, Vercel Functions

There is no machine you think about. Your function sits dormant, and each request wakes a copy of it.

- Costs nothing when nobody is using it
- Scales to enormous traffic without you doing anything
- **Cold starts:** the first call after a quiet period is slow
- Time limits per call — long jobs do not fit
- Nothing is remembered between calls; a local file is gone next time
- Awkward for a whole application; natural for one task

Right for a webhook, a scheduled job, or one endpoint called occasionally. Wrong for a first full-stack app, where the pieces need to sit together.

Side by side

	Static	**Platform**	Server	Serverless
You provide	files	**code**	nothing	a function
Runs your Python	no	**yes**	yes	yes
You manage the OS	no	**no**	**yes**	no
Can hold a secret	no	**yes**	yes	yes
Free tier	yes	**yes**	rarely	yes, generous
Sleeps when idle	no	**yes (free tier)**	no	yes
Setup effort	minutes	**minutes**	hours	moderate
Control	none	**some**	total	little

So why Render?

Platform hosting, because we need Python to run and a secret to stay hidden — which rules out static — and we do not want to spend the session administering Linux, which rules out a bare server. Serverless would work for one endpoint but fights us on a full application with a database.

Among platforms, Render because:

- The free tier needs **no credit card** — this matters for a student cohort
- One configuration file describes the whole service
- It reads directly from GitHub

Railway, Fly.io and PythonAnywhere would all work. Nothing in this workshop is Render-specific except the shape of one configuration file. If you already use another, use it.

When you would outgrow it

- Real, continuous traffic — the free tier's sleeping becomes unacceptable
- You need more memory or CPU than the free plan allows
- You need files to survive a redeploy (free plans wipe the disk)
- You need something unusual installed at system level
- At large scale, renting a plain server becomes cheaper

Moving is not dramatic. Your code does not change. The configuration around it does.

1.6 Things you will see but not touch today

HTTPS and the padlock

`https` rather than `http` means the traffic between browser and server is encrypted. Someone watching the network sees that you contacted the site, but not what was sent.

Every hosting provider in this workshop provides it automatically and free. On a bare server you would set it up yourself.

Anything handling a password, a payment, or personal data must use HTTPS. Browsers now mark plain `http` pages as insecure.

Domain names

`sensor-triage.onrender.com` is a **domain name**. Computers actually find each other by number (an IP address); domain names exist because people cannot remember numbers. The lookup system that translates one to the other is called **DNS**.

Your free address is a name underneath your provider's own domain. A domain of your own costs roughly ten to fifteen dollars a year, and you point it at your application by changing one DNS record. Not needed today.

Containers, and Docker

A **container** packages your code together with everything it needs to run: the right Python version, the right libraries, the right system tools.

The problem it solves is old and familiar: *"it works on my machine."* It works on yours because your machine has things the server does not. A container carries those things with it, so the code runs identically everywhere.

Docker is the tool most people use to build them. Container hosting — Google Cloud Run, AWS ECS, Kubernetes — sits between platform and server hosting: more control than a platform, less administration than a bare machine.

We do not need it today. Our platform builds the equivalent for us from `requirements.txt`. Learn it when you need to guarantee that something runs the same way in two places.

Why the server needs `requirements.txt`

Your laptop has FastAPI installed because you installed it. The server starts completely empty.

`requirements.txt` is a list of what to install. The server reads it, downloads each package, and only then starts your program. Without it, your code fails on its first `import`.

1.7 What an API is

A way for your program to ask another program a question over the internet, and get an answer back.

That is all. No more mysterious than a request and a response — because that is exactly what it is.

You have used APIs already without the word:

- A weather app asking a weather service for today's forecast
- A payment button asking a bank whether a card is valid
- A map showing live traffic
- Signing in to a site "with Google"

Why we need one for AI

The large AI models need far more memory and processing power than a laptop has. So your program does not run the model. It sends the question to a company that can, and reads the answer.

```
your program  ——— question ———>  a very large computer
your program  <——— answer ———   running the model
```

You send text. You get text back. Your own machine does almost no work.

> **Note:** smaller AI models *can* run on a laptop — Part 5 covers this. But they need several gigabytes of memory, which is more than free hosting provides. That trade-off is a real decision, and Part 5 explains it.

Two APIs, two directions

Worth being clear about, because the word gets used for both:

- Today you **use** an API — you call an AI company's service
- Today you also **build** one — your `/readings` endpoint is an API, and in Part 5 a microcontroller calls it exactly as you called the AI company's

Same idea, opposite ends.

1.8 What an API key is

A long secret string that identifies your account with a service.

```
sk-ant-api03-x8Kq2vN...
```

Why it deserves care

An API key behaves like **a bank card with no PIN**.

There is no second step, no confirmation, no approval email. Whoever holds it can use your account and spend your money. Possession is the entire security model.

So:

- It must never appear in your code
- It must never be sent to a browser
- It must never be uploaded to GitHub
- It must never be in a screenshot, a video, or a group chat

Where it goes instead

In a small file called `.env`, which stays on your computer and is never uploaded. Your program reads it at the moment it runs.

> **This is what an *environment variable* is:** a setting supplied to a program when it starts, rather than written inside it. The same code then runs unchanged on your laptop and on the server — each reads its own settings.

And now the two halves of Part 1 meet:

The frontend is public (1.1). An API key must stay secret (1.8). Therefore the key cannot live in the frontend.

It lives on your backend, which the visitor never sees. **That is what the backend is for.** If you only had a static page, you would have nowhere safe to put it.

```
   browser  ──>  your server  ──>  AI company
                 (holds the key)
```

The browser sends only the data. Your server adds the key and passes the question on. The key never travels to the visitor, so it can never be taken.

1.9 Where data lives

A file

We will use **SQLite** — a complete database inside a single file. Nothing to install, no server, no password. Your program opens the file and uses it.

For a prototype it is genuinely the right choice, and plenty of production software uses it.

A managed database

Postgres and **MySQL** are databases that run as their own service, on their own machine, that many programs can use at once. Hosting providers rent them by the month, and some offer a small free one.

Which, and when

	A file (SQLite)	A managed database
Setup	none	an account and a connection string
Cost	free	free tier, then monthly
Many programs at once	poor	designed for it
Survives a redeploy	**no, on free hosting**	yes

That last row is the one that matters, and it surprises people.

Free hosting wipes the disk every time you deploy. Your database file is part of that disk, so everything stored in it disappears when you push an update. The application still works; the history is empty.

For today that is fine — we are learning the shape, not running a business. If your project needs data to survive, that is the moment to add a managed database, and it is a small change.

1.10 The wider landscape

Everything in this workshop is one choice out of several at each layer. None of those choices is the only correct one, and knowing the alternatives is what lets you make your own decision on the next project.

For each layer: what exists, and why we picked what we picked.

The backend framework

What runs your server code.

	Language	Notes
FastAPI	Python	what we use. Automatic validation and an automatic test page
Flask	Python	older and simpler. No automatic validation or docs
Django	Python	batteries included: admin panel, user accounts, database layer. Heavier
Express	JavaScript	the default in the Node world

	Language	Notes
Spring Boot	Java	the enterprise standard
Gin, net/http	Go	compiled and very fast
Rails, Laravel	Ruby, PHP	mature and highly opinionated

Why FastAPI: you already write Python for AI and engineering work, so there is no second language to learn. The automatic validation and the free `/docs` page do real work for you, and neither Flask nor Express gives you those without extra libraries.

The frontend

What draws the page.

	Notes
Plain HTML and JavaScript	what we use. No build step, nothing to install, works everywhere
React, Vue, Svelte	component frameworks for large interfaces. Require a build step
Streamlit, Gradio	Python only, no HTML at all. Extremely fast for a dashboard
Flutter, React Native	mobile applications
Native iOS / Android	full platform access

Why plain HTML: the subject of this workshop is deployment, not interface design. A build step is one more thing that can break in front of an audience, and React would teach you React rather than teach you deployment.

> **The important point is underneath this table.** Every option above talks to the same backend, in the same way. Your `/readings` endpoint does not know or care whether the request came from plain JavaScript, a React application, a Flutter phone app, or a microcontroller.
> **Build the backend once, and every kind of client can use it.** That is why the split between frontend and backend exists at all.

Streamlit and Gradio, specifically

Worth calling out, because for some projects they are genuinely the better choice.

Both let you build a web interface in pure Python, with no HTML. A dashboard that would take you an hour here takes about fifteen lines there.

But neither one can receive a POST from a device. They have no route handlers. If your project has a microcontroller sending readings, you need a real backend. If it is software only and you just need a face on it, Streamlit will save you hours.

The database

	Type	Notes
SQLite	relational	what we use. A whole database in one file
PostgreSQL	relational	the general-purpose default for real applications
MySQL / MariaDB	relational	similar, extremely widely deployed
MongoDB	document	stores JSON-shaped records; flexible shape
Redis	key-value	in-memory and very fast; used for caching and queues

Three broad categories, and the difference matters more than the brand:

- **Relational** — data in tables with fixed columns, queried with SQL. Right when your data has a consistent shape, which most sensor data does.
- **Document** — records that are JSON objects and need not all match. Right when the shape varies or changes often.
- **Key-value** — a very fast lookup by name. Usually a cache in front of something else, not your main storage.

Why SQLite: no installation, no configuration, no account. When it stops being enough, PostgreSQL is the usual next step, and the SQL you learned still applies.

The AI provider

	Notes
Anthropic (Claude)	what we use
OpenAI	similar shape and pricing model
Google (Gemini)	similar; a free tier is often available
Mistral, Cohere	smaller providers, competitive pricing
Ollama, vLLM	run open models on your own machine or server
Hugging Face	thousands of open models, hosted or downloaded
AWS Bedrock, Azure AI, Vertex AI	the same models, billed through a cloud provider

They nearly all follow the same shape: send text plus a key, get text back. The library differs, the endpoint differs, the parameter names differ slightly. The idea does not.

That is exactly why our AI call lives alone in `ai.py`. Changing provider means editing one file, and nothing else in the application notices.

Automation, once your project is real

Not needed today, but you will meet these:

	What it does
Docker	packages your code with everything it needs, so it runs identically anywhere
GitHub Actions	runs tasks automatically on every push: tests, checks, deployment
pytest	writes tests that check your code still works after a change
Sentry, uptime monitors	tell you your application broke before your users do

How to choose, on your next project

Four questions, in this order:

1. **Does anything need to send data to it?** If yes, you need a real backend. That rules out static hosting, Streamlit and Gradio.
2. **Do you have a secret to protect?** If yes, you need a backend. Same conclusion.
3. **What language do you already write?** Use it. Learning a framework is quick; learning a language while learning deployment is not.
4. **How long does it need to live?** A demonstration, a term project, and a product have very different answers, and it is fine to choose the cheap option for the first two.

Almost every "which technology should I use" question is answered by one of those four.

1.11 What we are building

A **sensor triage service**.

Something sends a reading. Your code decides whether it is safe. An AI writes a short explanation. The result is stored and displayed.

Why this one

Because it is the shape every Project Nexus track shares: **a number, and a range it should stay inside.**

Track	Reading	Safe range
Healthcare	body temperature 39.4 °C	36.1 - 37.2 °C
Healthcare	blood oxygen 88 %	95 - 100 %
Sustainable Solutions	indoor CO2 1450 ppm	400 - 1000 ppm
Sustainable Solutions	power draw 4800 W	0 - 3500 W

Track	Reading	Safe range
Global Impact	soil moisture 8 %	30 - 70 %
Global Impact	motor temperature 87 °C	20 - 60 °C

Same code for all of them. Only the numbers change.

When several different problems share one shape, you write one program instead of several. Noticing that shape is a large part of software design.

And where it fits in this series

Every other workshop in this series produces something that lives on a laptop or on a device: a trained model, a program on a microcontroller, sensor readings in a terminal.

This is the session that puts those things somewhere other people can see them. The endpoint you build today accepts a reading from a browser — and it accepts exactly the same reading from an ESP32, with no change to your code.

Before you continue

You should now be able to answer these. If any is unclear, that section is worth re-reading before Part 2.

1. Why can a static host not keep an API key safe?
2. What is the difference between platform hosting and server hosting?
3. What does `:8000` mean in `127.0.0.1:8000`?
4. Why does the server need `requirements.txt` when your laptop does not?
5. Why must the API key live on the backend rather than in the page?
6. What happens to your SQLite file when you redeploy on free hosting?
7. Why can Streamlit not receive a reading from an ESP32?
8. Name one reason you would choose PostgreSQL over SQLite.

Part 2 is the fast path: a model out of a notebook and onto the internet in twenty minutes. **Part 3** then builds the full application.

Part 2 — The fast path

You will have something on the internet in about twenty minutes.

This part uses no HTML, no server code, and no deployment configuration. If you have never deployed anything, start here — you will have a working public link before we build the fuller version in Part 3.

If you are already comfortable with web development, read it anyway. Knowing when this path is enough will save you days.

2.1 The situation this solves

Most student projects end here:

```
a notebook that works   ->   on one laptop   ->   nobody else can use it
```

You trained a model. It works. It lives in a `.ipynb` file, and the only way anyone else sees it is if they sit next to you.

The fast path turns that into a web address, in three steps:

```
train_model.ipynb   ->   model.pkl   ->   app.py   ->   a public URL
   (train it)          (save it)      (wrap it)     (share it)
```

2.2 Step 1 — Get the model out of the notebook

The companion repository has `fastpath/train_model.ipynb`. Open it and run every cell.

It does four things, and they are the four things every training notebook does:

Cell	What it does
make data	3,000 example readings with known answers
choose a feature	*where* the reading sits in its safe range, as one number
train	a decision tree learns the pattern
save	**`joblib.dump(model, "model.pkl")`**

That last line is the important one.

```
joblib.dump(model, "model.pkl")
```

> **`model.pkl` IS the model.** It is a file, about two kilobytes, and it is the only thing that needs to leave your notebook. Everything above that line ran once and never runs again.
> This is the step people miss. A notebook is where you *make* a model. It is not where the model lives afterwards.

If you have your own trained model from another workshop, save it the same way and skip to the next step.

A note on the feature

The notebook does not give the model three raw numbers. It gives it one:

```
position = (value - low) / (high - low)
```

0.0 means the reading is at the bottom of its safe range, 1.0 at the top, negative means below it, above 1.0 means over it.

That single number lets one model handle a body temperature of 37 °C and a motor temperature of 87 °C without being confused by the difference in scale. Choosing what to feed a model matters more than which model you choose.

2.3 Step 2 — Wrap the model in an interface

Two libraries do this job. Both take a Python function and build a web page around it. Choose either; the model file and everything before this point is identical.

Option A — Gradio

Create `app.py` next to `model.pkl`:

```
import gradio as gr
import joblib

model = joblib.load("model.pkl")


def check_reading(value, low, high):
    """Runs when someone presses the button."""
    span = (high - low) or 1
    position = (value - low) / span            # the same feature we trained on
    severity = model.predict([[position]])[0]

    note = {
        "normal":   "Inside the safe range.",
        "warning":  "Outside the safe range. Worth checking.",
```

```
        "critical": "Far outside the safe range. Act now.",
    }[severity]
    return severity.upper(), note


demo = gr.Interface(
    fn=check_reading,
    inputs=[
        gr.Number(label="Reading", value=39.4),
        gr.Number(label="Safe range: lowest", value=36.1),
        gr.Number(label="Safe range: highest", value=37.2),
    ],
    outputs=[
        gr.Text(label="Severity"),
        gr.Text(label="What it means"),
    ],
    title="Sensor Triage",
    description="Enter a reading and its safe range. A trained model decides how serious it i
    examples=[
        [36.8, 36.1, 37.2],
        [39.4, 36.1, 37.2],
        [1450, 400, 1000],
        [8, 30, 70],
    ],
)

if __name__ == "__main__":
    demo.launch()
```

```
pip install gradio scikit-learn joblib
```

```
python app.py
```

Open http://127.0.0.1:7860.

Option B — Streamlit

Create `streamlit_app.py`:

```
import joblib
import streamlit as st

model = joblib.load("model.pkl")

SEVERITY_NOTE = {
    "normal":   "Inside the safe range.",
    "warning":  "Outside the safe range. Investigation recommended.",
```

```
    "critical": "Far outside the safe range. Immediate action required.",
}
SEVERITY_COLOUR = {"normal": "green", "warning": "orange", "critical": "red"}

st.title("Sensor Triage")
st.caption("Enter a reading and its safe operating range. "
           "A trained model determines the severity.")

value = st.number_input("Reading", value=39.4)
low = st.number_input("Safe range: lowest", value=36.1)
high = st.number_input("Safe range: highest", value=37.2)

# st.session_state survives re-runs, so the table below persists while the
# browser tab is open. It is NOT a database: refresh the page and it is gone.
if "history" not in st.session_state:
    st.session_state.history = []

if st.button("Check reading", type="primary"):
    span = (high - low) or 1
    position = (value - low) / span          # the same feature used in training
    severity = model.predict([[position]])[0]

    st.markdown(f"### :{SEVERITY_COLOUR[severity]}[{severity.upper()}]")
    st.write(SEVERITY_NOTE[severity])
    st.caption(f"position in range: {position:.2f}  "
               f"(0.0 = lowest, 1.0 = highest)")

    st.session_state.history.insert(0, {
        "Reading": value,
        "Safe range": f"{low} - {high}",
        "Position": round(position, 2),
        "Severity": severity,
    })

if st.session_state.history:
    st.divider()
    st.subheader("This session")
    st.dataframe(st.session_state.history, use_container_width=True, hide_index=True)
    st.caption("Held in the browser session only. Refresh the page and it is lost. "
               "Storing readings properly requires a database, which is Part 3.")
```

```
pip install streamlit scikit-learn joblib
```

```
streamlit run streamlit_app.py
```

Open http://localhost:8501.

> **`st.session_state` is not a database.** It keeps the table while the browser tab is open, and loses it on refresh. That is enough to demonstrate a series of readings, and it is deliberately not enough to run a system on. Storing readings properly is Part 3.

The difference between them

	Gradio	Streamlit
Style	describe inputs and outputs, pass a function	write the page top to bottom
Best suited to	one function with fixed inputs — a model	dashboards, charts, several sections
Reruns	only your function, on submit	the whole script, on every interaction
Free hosting	Render, or Hugging Face Spaces (paid, see below)	**Streamlit Community Cloud**

Both are good. **Streamlit currently has the better free hosting route**, which is the deciding factor for this workshop.

> Note the `position` calculation appears in the training notebook and again in the application. Whatever you compute before training, you must compute again before predicting. Getting this wrong produces an application that runs perfectly and returns nonsense.

2.4 Step 3 — Deploy it

The current free options

Checked August 2026. **Verify before you rely on any of them** — free tiers change, sometimes without notice.

	Runs	Free	Requires
Streamlit Community Cloud	Streamlit	**yes**	a GitHub repository
Render	either	**yes**	a GitHub repository
Hugging Face Spaces — Static	HTML only	yes	cannot run Python
Hugging Face Spaces — Gradio	Gradio	**no, paid plan**	—
Hugging Face Spaces — Docker	anything	**no, paid plan**	—

> **Hugging Face Spaces changed.** It was the simplest route for a Gradio application: upload three files through a web page, no repository needed. Gradio and Docker Spaces now require a paid subscription. Only Static Spaces remain free, and those cannot run Python, so they cannot run a model.
> This is worth noticing as a general lesson rather than an inconvenience. **Free tiers move.** Anything you build should survive its host changing terms — which is an argument for keeping your model and your logic separate from whatever wraps them.

Deploying to Streamlit Community Cloud

1. Put these three files in a GitHub repository:

```
streamlit_app.py     your interface
model.pkl            your trained model
requirements.txt     what to install
```

`requirements.txt`:

```
streamlit
scikit-learn
joblib
```

2. Go to **https://share.streamlit.io** and sign in with GitHub
3. Click **Create app**, then **Deploy a public app from GitHub**
4. Choose your repository, branch, and `streamlit_app.py` as the main file
5. Click **Deploy**

The build takes two to three minutes. Your application is then live at an address ending in `.streamlit.app`.

Limits: approximately 1 GB of memory, and the application sleeps after about 12 hours without visitors. That sleep window is considerably more forgiving than most free hosting, which is useful for an exhibition.

Deploying Gradio instead

Gradio is an ordinary Python web application, so any platform hosting Python will run it — including Render, which we use in Part 4. Add a `render.yaml` with:

```
startCommand: python app.py
```

and set `server_name="0.0.0.0"` and `server_port` from the `PORT` environment variable in `demo.launch()`. Part 4 covers what those two settings mean.

2.5 What this path cannot do

Everything above is real, and for some projects it is all you need. Be clear about the ceiling before you commit to it.

	Fast path (Gradio or Streamlit)	Full path (Part 3)
Time to a public URL	**20 minutes**	about an hour
HTML or server code	**none**	some
A person can use it	**yes**	yes
A device can send readings to it	**no**	**yes**
Custom addresses your code controls	no	yes
Store results between visits	no	yes
Control how it looks	limited	complete
Another program can call it	awkward	yes, it is an API

The row that decides it is the middle one.

Both Gradio and Streamlit build a page for a *person* to fill in. Neither gives you an address a microcontroller can POST to. If your Project Nexus prototype has an ESP32 sending sensor readings, this path cannot receive them, and no amount of configuration changes that.

So which should you use?

Use the fast path if:

- your project is a model, and people will type inputs into it
- you need something online today
- you are showing a capability rather than running a system

Use the full path if:

- hardware sends data to it
- you need to store results
- another program needs to call it
- you want control over the interface

Many teams should do both. The fast path is a good demonstration of your model on its own. The full path is your actual prototype. They are not in competition, and building the first one takes twenty minutes.

2.6 What you have now

```
train_model.ipynb  ->  model.pkl  ->  app.py  ->  a public URL
```

You have taken a model out of a notebook and put it somewhere other people can reach. That is the whole gap this workshop exists to close, and you have now closed it once.

Part 3 closes it the other way — building a service that a device can talk to, that stores what it receives, and that you control completely.

Part 3 — Build it

Five stages. **Each one ends with something you can see on screen**, and each builds on the last.

Work in **one folder of your own** the whole way through. The `stages/` folder in this repository holds a complete working copy of every stage — if you fall behind, copy the one you need and carry on.

Before you start — create the project

Make the folder

```
mkdir sensor-triage
```

```
cd sensor-triage
```

Create a virtual environment

```
python -m venv .venv
```

> **What is a virtual environment?** A private copy of Python for this project alone. Packages you install here do not affect your other projects, and other projects cannot break this one.
> It creates a `.venv` folder. You never edit anything inside it.

Activate it

Windows:

```
.venv\Scripts\activate
```

macOS / Linux:

```
source .venv/bin/activate
```

Your prompt now starts with `(.venv)`. That is how you know it is on.

> **If PowerShell says "running scripts is disabled on this system":** that is the Windows trap from Part 0. Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer `Y`, and try again.

You must activate it every time you open a new terminal. Forgetting is the most common cause of "but I installed it already".

Install what we need

```
pip install "fastapi[standard]" uvicorn python-dotenv anthropic
```

Package	What it is for
`fastapi`	the framework — turns Python functions into web addresses
`uvicorn`	the program that actually runs your server
`python-dotenv`	reads your settings file, from Stage 5
`anthropic`	talks to the AI service, from Stage 5

Open the folder in VS Code

```
code .
```

The dot means “this folder”. If that command does not work, use **File -> Open Folder** and choose `sensor-triage`.

Stage 1 — A server that answers

Goal: a program running on your machine that replies to a browser.

Create `main.py`

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/health")
def health():
    return {"status": "ok"}
```

That is the whole file. Six lines.

Run it

```
uvicorn main:app --reload
```

You should see:

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Application startup complete.
```

> **What does `main:app` mean?** The file called `main`, and the object called `app` inside it. If you named your file something else, that first word changes.
> **What does `--reload` do?** Restarts the server automatically whenever you save a file. Leave it running for the whole workshop.

See the result

Open **http://127.0.0.1:8000/health**

```
{"status":"ok"}
```

You are running a web server. Something on your machine received a request and sent back an answer.

Now open the free test page

Open **http://127.0.0.1:8000/docs**

FastAPI generated an interactive page for your API automatically, from your code. Expand `/health`, click **Try it out**, then **Execute**.

You will use this page for the next two stages. It means you can test your backend without writing a single line of frontend.

New terms

> **Endpoint** — one address your server answers, and the code behind it. `/health` is an endpoint. An application is a collection of them.

> **Decorator** — the `@app.get("/health")` line. It tells FastAPI: when a GET request arrives for this address, run the function underneath.

Why `/health` in particular

Hosting platforms call an address like this every few seconds to check your application is alive, and restart it if not. We point Render at it in Part 4.

Almost every production service has one. Yours now does too.

Stage 2 — The page appears

Goal: a real web page, that talks to the server you just built.

Create the folder and file

Inside `sensor-triage`, create a folder called `static`, and inside it a file called `index.html`.

The full file is at **`stages/stage2/static/index.html`** — copy it. The part that matters is this:

```
// Ask our own server whether it is alive.
// This is a request; what comes back is a response.
async function checkServer() {
  const box = document.getElementById("status");
  try {
    const res  = await fetch("/health");
    const data = await res.json();
    box.className   = "ok";
    box.textContent = "Connected to your server. It replied: " + data.status;
  } catch (err) {
    box.className   = "fail";
    box.textContent = "Could not reach the server.";
  }
}
checkServer();
```

That is the frontend calling the backend. `fetch("/health")` sends a request to the same endpoint you tested in `/docs` a moment ago — this time from a page instead of from a form.

Serve it

Add one import at the top of `main.py`:

```
from fastapi.staticfiles import StaticFiles
```

And add this at the **very bottom** of the file:

```
# Serve the web page from the "static" folder.
# This line must always be LAST, after every endpoint above it.
app.mount("/", StaticFiles(directory="static", html=True), name="static")
```

> **This line must be last. Every time.**
>
> `app.mount("/")` claims every address that has not already been claimed. Put it above your endpoints and it swallows them — `/health` would start returning your HTML page instead of data.
> FastAPI matches routes in the order you wrote them, and first match wins. **Rule to remember: the mount line is always the last line in the file.**

See the result

Open **http://127.0.0.1:8000**

> **Sensor Triage**
>
> Stage 2 — the page can now talk to your server.
> **Connected to your server. It replied: ok**

That sentence is proof of the whole Part 1 model. Your browser made a request. Your Python code produced a response. The page displayed it.

What just happened

```
your browser  ——— GET /health ———>  your Python code
your browser  <———  {"status":"ok"} ——  your Python code
```

Three separate things you wrote are now working together: a page, a server, and the connection between them. Everything from here is adding to that.

Stage 3 — Your code decides

Goal: send a real reading, and get a verdict back.

Add the input description

At the top of `main.py`, add this import:

```
from pydantic import BaseModel
```

Then add this class above your endpoints:

```
class Reading(BaseModel):
    """Describes what a valid reading looks like.

    FastAPI checks every incoming request against this. If a field is
    missing or the wrong type, the request is rejected before your code runs.
    """
    sensor: str
    value: float
    unit: str
    low: float
    high: float
```

This is a **description**, not code that runs. You are telling FastAPI the shape you expect: what the sensor is, what it read, the unit, and the two ends of the safe range.

Add the rule

```
def classify(value: float, low: float, high: float) -> str:
    """Decide how serious a reading is. Plain arithmetic, no AI."""
    if low <= value <= high:
        return "normal"

    margin = (high - low) / 2 or 1      # how far outside counts as "a lot"
    if value < low - margin or value > high + margin:
        return "critical"

    return "warning"
```

Read it aloud: inside the range is normal, a long way outside is critical, otherwise warning.

The `margin` line is the only clever part. Half the width of the safe range counts as "a lot" — for a body temperature range of 36.1-37.2 that is about half a degree; for a motor range of 20-60 it is twenty

degrees. **The rule scales itself to whatever is being measured.**

> **Remember this function.** In Stage 5 we explain why it stays plain arithmetic and never becomes the AI's job.

Add the endpoint

```
@app.post("/readings")
def create_reading(reading: Reading):
    severity = classify(reading.value, reading.low, reading.high)
    return {
        "sensor": reading.sensor,
        "value": reading.value,
        "unit": reading.unit,
        "severity": severity,
    }
```

Two things are new. `@app.post` rather than `@app.get`, because this request carries data. And `reading: Reading` in the signature — **that annotation is what tells FastAPI to read the request body, check it against your class, and hand you a finished object.** You never parse anything yourself.

Update the page

Copy `stages/stage3/static/index.html` over your existing file. It adds a scenario picker, a reading box, and a Send button. The part that matters:

```
// Send the reading to our own server and wait for its answer.
const res = await fetch("/readings", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    sensor: current.sensor, value,
    unit: current.unit, low: current.low, high: current.high,
  }),
});
```

See the result

Open **http://127.0.0.1:8000**, choose a scenario, press **Send**.

> **CRITICAL** your code checked: 36.1 <= 39.4 <= 37.2

Try several. These are the answers you should get:

Reading	Safe range	Result
36.8 °C	36.1 - 37.2	`normal`
37.4 °C	36.1 - 37.2	`warning`

Reading	Safe range	Result
39.4 °C	36.1 - 37.2	`critical`
88 %	95 - 100	`critical`
1450 ppm	400 - 1000	`critical`

Now break it on purpose

Go to `/docs`, find **POST /readings**, click **Try it out**, and send text where a number belongs:

```
{ "sensor": "motor_temp", "value": "hot", "unit": "C", "low": 20, "high": 60 }
```

You get back **HTTP 422**, with a message naming the bad field — and **your `classify` function never ran.** The `Reading` class rejected the request at the door.

> **Never trust incoming data**
>
> Once your address is public, anyone can send anything to it: a sensor with a loose wire, a teammate's typo, or a stranger deliberately probing your service.
> Declaring the shape you expect defends against all three at once — and you got it from one class definition, not from validation code you had to write.
> This is a habit worth keeping: **describe your input at the edge of your application, every time.**

New terms

> **JSON** — the text format programs use to exchange data. It looks like a Python dictionary and behaves like one. Your browser, your Python code, and a microcontroller all speak it, which is why they can talk to each other.

> **GET and POST** — GET asks for something; POST sends something. Reading a page is a GET, submitting a form is a POST.

Stage 4 — Remember the readings

Right now every reading is forgotten the moment you reply. Let's keep them.

Goal: readings that survive a restart, listed on the page.

Add the database helper

New imports at the top:

```
import sqlite3
from contextlib import contextmanager
```

Then, just after `app = FastAPI()`:

```
DB = "readings.db"


@contextmanager
def db():
    """Open the database, hand it over, then always close it."""
    conn = sqlite3.connect(DB)
    conn.row_factory = sqlite3.Row      # lets us read columns by name
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()
```

The `finally` block is the point: whatever happens inside, the connection gets closed. A connection left open is a resource leak, and in a server that runs for weeks it eventually becomes a crash. Writing it once here makes every use safe.

Create the table

```
def init_db():
    with db() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS readings (
                id       INTEGER PRIMARY KEY AUTOINCREMENT,
                created  TEXT DEFAULT CURRENT_TIMESTAMP,
                sensor   TEXT,
                value    REAL,
                unit     TEXT,
```

```
                low      REAL,
                high     REAL,
                severity TEXT
            )
        """)


init_db()
```

> **A trap you will hit in Stage 5**
>
> `CREATE TABLE IF NOT EXISTS` will **not** change a table that already exists.
> In Stage 5 we add two columns. When you do, you must **delete `readings.db`** and restart, or you will get an error about a missing column and no explanation of why.

Save each reading

Replace the body of `create_reading`:

```python
@app.post("/readings")
def create_reading(reading: Reading):
    severity = classify(reading.value, reading.low, reading.high)

    with db() as conn:
        cursor = conn.execute(
            "INSERT INTO readings (sensor, value, unit, low, high, severity)"
            " VALUES (?, ?, ?, ?, ?, ?)",
            (reading.sensor, reading.value, reading.unit,
             reading.low, reading.high, severity),
        )
        new_id = cursor.lastrowid

    return {
        "id": new_id,
        "sensor": reading.sensor,
        "value": reading.value,
        "unit": reading.unit,
        "severity": severity,
    }
```

> **Why the question marks?**
>
> `VALUES (?, ?, ?)` with the values passed separately is a **parameterised query**. The database receives the command and the data as two different things.
> **Never build SQL by joining strings together.** That is how SQL injection attacks work: someone sends a "sensor name" that is actually a command, and your database runs it. The question marks make that impossible, and they cost nothing.

Add an endpoint to read them back

```python
@app.get("/readings")
def list_readings(limit: int = 20):
    with db() as conn:
        rows = conn.execute(
            "SELECT * FROM readings ORDER BY id DESC LIMIT ?", (limit,)
        ).fetchall()
    return [dict(row) for row in rows]
```

Note that `/readings` now has **two** endpoints: a POST that stores, and a GET that lists. Same address, different methods, different code. That is normal and deliberate.

Update the page

Copy `stages/stage4/static/index.html`. It adds a table underneath, and this:

```javascript
// Fetch everything the server has stored and draw the table.
async function loadHistory() {
  const rows = await (await fetch("/readings")).json();
  ...
}
```

See the result

Send three or four readings. They now appear in a table below.

Time	Reading	Severity
08:06:46	co2 1450ppm	**critical**
08:06:46	spo2 97%	**normal**
08:06:46	body_temp 39.4C	**critical**

Now prove it is real

1. Stop the server — `Ctrl + C`
2. Start it again — `uvicorn main:app --reload`
3. Refresh the page

The readings are still there. And a new file called `readings.db` has appeared in your folder. That file *is* the database.

> Do this comparison if you can: open Stage 3 and Stage 4 side by side, send readings to both, restart both, and refresh. Stage 3 is blank. Stage 4 is not. That is what a database is, in one move.

New term

> **SQLite** — a complete database inside a single file. Nothing to install, no server to run, no password. For a prototype it is genuinely the right choice, and a great deal of production software uses it.

Stage 5 — The AI explains

Goal: a written explanation alongside every verdict.

First, a design decision

Before any code. This is the most useful idea in the workshop.

Look at `classify()`. It compares numbers. It is instant, free, correct every time, and you can write a test for it.

Do not give that job to an AI.

Tested on the same eight readings, three runs each, asking only *"is this inside the safe range?"*:

Approach	Correct
Plain Python arithmetic	**100 %**
A small language model	38 %
A larger language model	83 %

On a yes-or-no question, guessing scores about 50 %. The small model did worse than guessing, because it has a habit of raising the alarm even when everything is fine.

And the larger one classified **87 °C as "normal"** when the safe range was 20-60 °C — on two runs out of three.

> A false negative is the worst kind of error here. Saying "critical" when things are fine is annoying. Saying "normal" when the motor is overheating is the failure that matters.
> It is also **not repeatable**. The same question gave different answers. You cannot write a passing test for something that changes its mind.

So we split the work

	Your code decides	The AI explains
Produces	the severity	the sentence
Speed	instant	seconds
Correct?	always, by definition	usually
Testable?	yes	not really
If it fails	it cannot	the app still works

Comparing two numbers is not a job for a language model. Writing a sentence a tired engineer wants to read at 2am is — and that is something code genuinely cannot do.

This is not "AI is bad". It is ordinary engineering judgement: know what each component is good at, and do not route work to the wrong one.

Now build it

Create `.env`

A file called exactly `.env` in your project folder:

```
# Copy this file to .env and edit it. Never commit .env
#
# Which AI service to use:
#   mock       no key needed, canned replies. Good for building and testing.
#   anthropic  Claude
#   openai     ChatGPT
#   deepseek   DeepSeek
#   gemini     Google Gemini
#   groq       Groq
#   ollama     a model running on your own machine (no key needed)
AI_PROVIDER=mock

# Only the one matching AI_PROVIDER is read.
ANTHROPIC_API_KEY=
OPENAI_API_KEY=
DEEPSEEK_API_KEY=
GEMINI_API_KEY=
GROQ_API_KEY=

# Optional. Each provider has a sensible default.
# ANTHROPIC_MODEL=claude-haiku-4-5
# OPENAI_MODEL=gpt-4o-mini
# DEEPSEEK_MODEL=deepseek-chat
```

> **Windows:** if you did not turn on file extensions in Part 0, Notepad will save this as `.env.txt` and your program will not find it. Check the filename.

Create `.gitignore`

```
.env
*.db
__pycache__/
.venv/
```

> **Create these two files now, before there is ever a real key to lose**
>
> `.env` holds secrets. `.gitignore` is the list of files that must never be uploaded, and `.env` is the first line for a reason.
> Most people add `.gitignore` *after* they already have a key sitting in the folder. Doing it in this order is the habit worth building.

Create `ai.py`

Everything to do with the AI lives in its own file — including the key.

```
"""Everything to do with the AI provider lives in this one file.

It is also the only file that ever touches an API key.

Set AI_PROVIDER in .env to choose. Nothing else in the application changes.
"""
import json
import os

from pydantic import BaseModel, Field

PROVIDER = os.getenv("AI_PROVIDER", "mock").lower()

SYSTEM = (
    "You explain sensor readings to a maintenance engineer. "
    "Be specific and brief. Always mention the measured value."
)


class Advice(BaseModel):
    """The shape of the answer we require, whichever provider produces it."""
    reason: str = Field(description="why the reading is at this level, under 15 words")
    action: str = Field(description="one concrete next step, under 10 words")


# Providers that speak the OpenAI protocol. Only the address and the model
# name differ, which is why one function serves all of them.
OPENAI_COMPATIBLE = {
    "openai":   ("https://api.openai.com/v1",           "OPENAI_API_KEY",   "gpt-4o-mini"),
    "deepseek": ("https://api.deepseek.com/v1",         "DEEPSEEK_API_KEY", "deepseek-chat"),
    "gemini":   ("https://generativelanguage.googleapis.com/v1beta/openai/",
                 "GEMINI_API_KEY",  "gemini-2.0-flash"),
    "groq":     ("https://api.groq.com/openai/v1",      "GROQ_API_KEY",     "llama-3.3-70b-ve
```

```
    "ollama":   ("http://localhost:11434/v1",          "OLLAMA_API_KEY",   "llama3.2:3b"),
}


def _prompt(sensor, value, unit, low, high, severity):
    return (
        f"A {sensor} sensor reads {value}{unit}. "
        f"Its safe range is {low}{unit} to {high}{unit}. "
        f"An automatic check rated this {severity}."
    )


def _mock(sensor, value, unit, low, high, severity):
    return Advice(
        reason=f"{value}{unit} against a safe range of {low}-{high}{unit}.",
        action="Set AI_PROVIDER to a real provider for a written answer.",
    )


def _anthropic(sensor, value, unit, low, high, severity):
    """Anthropic's own SDK. output_format guarantees the two fields."""
    import anthropic

    client = anthropic.Anthropic()      # reads ANTHROPIC_API_KEY
    response = client.messages.parse(
        model=os.getenv("ANTHROPIC_MODEL", "claude-haiku-4-5"),
        max_tokens=256,
        system=SYSTEM,
        messages=[{"role": "user",
                   "content": _prompt(sensor, value, unit, low, high, severity)}],
        output_format=Advice,
    )
    return response.parsed_output


def _openai_compatible(sensor, value, unit, low, high, severity):
    """OpenAI, DeepSeek, Gemini, Groq and a local Ollama all speak this."""
    from openai import OpenAI

    base_url, key_name, default_model = OPENAI_COMPATIBLE[PROVIDER]
    api_key = os.getenv(key_name) or "not-needed"      # Ollama needs no key
    model = os.getenv(f"{PROVIDER.upper()}_MODEL", default_model)

    client = OpenAI(base_url=base_url, api_key=api_key)
    completion = client.chat.completions.create(
```

```
        model=model,
        max_tokens=256,
        response_format={"type": "json_object"},       # ask for JSON, not prose
        messages=[
            {"role": "system",
             "content": SYSTEM + ' Reply as JSON: {"reason": "...", "action": "..."}'},
            {"role": "user",
             "content": _prompt(sensor, value, unit, low, high, severity)},
        ],
    )
    return Advice(**json.loads(completion.choices[0].message.content))


def explain(sensor: str, value: float, unit: str,
            low: float, high: float, severity: str) -> Advice:
    """Ask the configured provider to put the reading into words."""
    if PROVIDER == "mock":
        return _mock(sensor, value, unit, low, high, severity)
    if PROVIDER == "anthropic":
        return _anthropic(sensor, value, unit, low, high, severity)
    if PROVIDER in OPENAI_COMPATIBLE:
        return _openai_compatible(sensor, value, unit, low, high, severity)

    raise RuntimeError(
        f"Unknown AI_PROVIDER '{PROVIDER}'. Use one of: "
        f"mock, anthropic, {', '.join(OPENAI_COMPATIBLE)}"
    )
```

Three things worth noticing

1. No client is given a key in code. Each one reads it from the environment by itself. **A key appears nowhere in your source.**

2. `output_format=Advice` guarantees the reply. The model cannot return a paragraph, a refusal, or a differently named field. You get a validated object back. Without this you would be writing code to find the useful part of some prose, and that code breaks constantly.

3. One setting switches provider. `AI_PROVIDER=mock` runs with no key at all. `anthropic`, `openai`, `deepseek`, `gemini`, `groq` and a local `ollama` all work from the same application — because OpenAI, DeepSeek, Gemini and Groq share one protocol, so a single function serves all four with a different address.

This is the seam paying off. Changing provider edits one file. Nothing else in the application notices.

Wire it into `main.py`

At the **very top**, above the other imports:

```
from dotenv import load_dotenv
load_dotenv()          # read .env BEFORE anything else looks at the environment
```

> That looks wrong — imports are normally grouped together. It is deliberate. `load_dotenv()` must run before anything else reads the environment, and `import ai` reads it immediately.

With the other imports:

```
import ai
```

Add two columns to the table in `init_db()`:

```
                severity TEXT,
                reason   TEXT,
                action   TEXT
```

That alone is not enough. `CREATE TABLE IF NOT EXISTS` will not alter a table that already exists, and yours does — you created it in Stage 4. So add this inside `init_db()`, just after the `CREATE TABLE` statement:

```
        # A database created at Stage 4 has no reason or action column.
        # CREATE TABLE IF NOT EXISTS will not add them, so add them here.
        existing = {row["name"] for row in conn.execute("PRAGMA table_info(readings)")}
        for column in ("reason", "action"):
            if column not in existing:
                conn.execute(f"ALTER TABLE readings ADD COLUMN {column} TEXT")
```

> **This is a migration, and every project eventually needs one**
>
> Your table already holds data. You cannot simply recreate it, and the `CREATE TABLE` statement will not change it. `ALTER TABLE` adds the missing columns to the table that is already there, leaving the existing rows intact.
> Without this you get:
> `sqlite3.OperationalError: table readings has no column named reason`
> which names the problem precisely and gives no hint of the cause.
> **The alternative is to delete `readings.db` and let it be recreated.** That is fine while you are learning and loses everything, which is why real applications migrate instead.

And update `create_reading`:

```
@app.post("/readings")
def create_reading(reading: Reading):
    # 1. Our own code decides how serious it is. Instant, always correct.
    severity = classify(reading.value, reading.low, reading.high)

    # 2. The AI turns that into a sentence a person can read.
    advice = ai.explain(reading.sensor, reading.value, reading.unit,
                        reading.low, reading.high, severity)
```

```
    with db() as conn:
        cursor = conn.execute(
            "INSERT INTO readings (sensor, value, unit, low, high,"
            " severity, reason, action) VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
            (reading.sensor, reading.value, reading.unit, reading.low,
             reading.high, severity, advice.reason, advice.action),
        )
        new_id = cursor.lastrowid

    return {
        "id": new_id,
        "sensor": reading.sensor,
        "value": reading.value,
        "unit": reading.unit,
        "severity": severity,
        "reason": advice.reason,
        "action": advice.action,
    }
```

Those two numbered comments are the design decision, written into the code.

Update the page

Copy `stages/stage5/static/index.html`. It adds the explanation under the verdict, and an Explanation column to the table.

See the result

> **CRITICAL** your code checked: 36.1 <= 39.4 <= 37.2
> 39.4C against a safe range of 36.1-37.2C. *Set USE_MOCK=false to get a real AI answer.*

With `USE_MOCK=true`, the application is complete and working with no AI account at all.

Using a real key

If you have one, set the provider and its key in `.env`, then restart:

```
AI_PROVIDER=anthropic
ANTHROPIC_API_KEY=sk-ant-api03-...
```

or

```
AI_PROVIDER=deepseek
DEEPSEEK_API_KEY=sk-...
```

Send another reading. The explanation is now written by the model.

What it costs. A request like this is roughly 500 tokens in, 150 out.

Model	Per request	Requests per $5
`claude-haiku-4-5`	$0.00125	about 4,000
`claude-opus-5`	$0.00625	about 800

Two pieces of advice: set a spending limit on your account before you start, and keep `USE_MOCK=true` while developing so you are not paying for every test run.

What you have now

```
sensor-triage/
├─ main.py            your server: three endpoints
├─ ai.py              the AI call, and the only file that sees the key
├─ .env               your settings and secrets      (never uploaded)
├─ .gitignore         the list of things not to upload
├─ readings.db        the database                    (never uploaded)
└─ static/
   └─ index.html      the web page
```

A working full-stack application:

- **Frontend** — a page that sends readings and displays results
- **Backend** — three endpoints, with input validation
- **Data** — a database that survives restarts
- **An external service** — an AI API, with the key handled correctly

It runs on your laptop and nowhere else. **Part 4 puts it on the internet.**

Part 4 — Put it on the internet

Your application works. It works on exactly one computer, and only while that computer is awake with a terminal open.

This part moves it to a machine that never sleeps, at an address anyone can open.

4.1 What "deploying" actually means

Three things have to happen.

1. Your code has to get there	you cannot email a folder to a server. This is what GitHub is for
2. The server has to know how to run it	it starts empty: no Python packages, no start command
3. It has to keep running	if it crashes at 3am, something must restart it

We solve them in that order: two small files, then GitHub, then Render.

Nothing about your application changes. `main.py`, `ai.py` and `index.html` are finished. Everything in this part is **around** your code, not inside it.

4.2 The two files a server needs

`requirements.txt` — what to install

Your laptop has FastAPI because you installed it. The server starts completely empty. This file is the shopping list.

```
fastapi==0.141.1
uvicorn[standard]==0.34.0
pydantic==2.10.4
python-dotenv==1.0.1
anthropic==1.0.0
```

To see your own exact versions:

```
pip freeze
```

> **Pin the versions**
>
> Writing `fastapi` with no == version means the server installs whatever is newest **on the day it builds**. Your application then breaks weeks later without you touching a single line of code. This is not hypothetical. While preparing this workshop, installing an unrelated package upgraded a library underneath FastAPI and broke the application. `requirements.txt` had pinned FastAPI, but not the library beneath it.
> Pin everything. `pip freeze` gives you the exact list.

`render.yaml` — how to run it

```yaml
# Render reads this file and sets the whole service up for you.
services:
  - name: sensor-triage
    type: web
    runtime: python
    plan: free                    # no credit card required
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
    healthCheckPath: /health      # Render calls this to check the app is alive
    envVars:
      - key: AI_PROVIDER
        value: "mock"             # change to a provider once you have added a key

      # 'sync: false' means: this setting exists, but its value is typed
      # into the Render dashboard, never written in this file.
      # This file is uploaded to GitHub. A key must never be in it.
      - key: ANTHROPIC_API_KEY
        sync: false
```

Line by line:

`type: web`	this service receives web requests
`runtime: python`	it is a Python project
`plan: free`	the free tier
`buildCommand`	runs **once**, when the code arrives
`startCommand`	runs to keep the application alive
`healthCheckPath`	the address Render calls to check you are alive

`/health` was worth writing in Stage 1 after all. Render will call it every few seconds, and restart your service if it stops answering.

Three details worth understanding

`--host 0.0.0.0`

On your laptop, uvicorn listens only for requests from your own machine. On a server that would make it unreachable. `0.0.0.0` means *accept requests from anywhere*.

`--port $PORT`

The hosting platform chooses the port and tells your application through an environment variable.

Hardcode `8000` here and you get a confusing failure: the logs look healthy, the service says it started, and the address shows nothing — because the platform is sending traffic to a door your app is not standing behind.

`sync: false`

This is the important one.

`render.yaml` gets **uploaded to GitHub**. A key written as a `value:` here is published exactly as if you had pasted it into your source code.

`sync: false` says: *this setting exists, but its value is not in this file.* You type the value into Render's dashboard afterwards, where it is private.

> **Any file that gets committed is a place a secret can hide.** `.env` is not the only one. Configuration files, CI workflow files, notebooks, Docker files. Build the habit of asking *"is this file uploaded?"* before typing a secret anywhere.

4.3 Git, in five minutes

You installed git in Part 0. Here is what it actually does.

The four words you need

> **Repository** (**repo**) — one project's folder, with all of its history. Your `sensor-triage` folder becomes one.

> **Commit** — a saved checkpoint. It records exactly what every file looked like at that moment, with a message saying what changed.

> **Remote** — a copy of your repository somewhere else. Ours will live on GitHub.

> **Push** — send your commits to the remote.

The mental model

```
your folder  ──commit──>  local history  ──push──>  GitHub
```

Git records checkpoints on **your machine** first. Pushing copies them somewhere else. That is why git works perfectly well offline, and why GitHub is optional right up until you need another computer to see your code.

Why we need it here

Render does not want a zip file. It connects to GitHub, reads your repository, and rebuilds your application from it. Every time you push, it can redeploy automatically.

4.4 Put your code on GitHub

Create the repository on GitHub

1. Go to **https://github.com/new**
2. **Repository name** — `sensor-triage` is fine
3. Choose **Public** or **Private** — either works with Render
4. **Do not tick** "Add a README file", ".gitignore", or "Choose a licence"

> Those three tick-boxes create files on GitHub, which conflicts with the files you already have locally, and produces a confusing error on your first push. Start empty.

5. Click **Create repository**

GitHub now shows a page of setup commands. We use them below.

Turn your folder into a repository

Back in your terminal, in your `sensor-triage` folder:

```
git init
```

```
git add .
```

`git add .` stages everything in the folder — except whatever `.gitignore` excludes.

Check what you are about to upload

This is the most important command in the whole workshop.

```
git status
```

Read the list carefully. You should see `main.py`, `ai.py`, `static/`, `requirements.txt`, `render.yaml`, `.gitignore`.

> **`.env` must NOT be in that list**
>
> Nor should `readings.db`, `.venv`, or `__pycache__`.
> If `.env` appears, **stop**. Your `.gitignore` is missing or misspelled. Fix it, run `git add .` again, and check again.
> Three seconds here prevents the problem in the next section, which has no clean fix.

Make your first commit

```
git commit -m "Sensor triage service"
```

The `-m` is the message. Write what changed, for the version of you that reads it in three months.

Connect it to GitHub and push

Replace the URL with your own — GitHub showed it on the page after you created the repository:

```
git remote add origin https://github.com/YOUR-USERNAME/sensor-triage.git
```

```
git branch -M main
```

```
git push -u origin main
```

The first time, a window may open asking you to sign in to GitHub. Do so; your computer remembers it afterwards.

Refresh your repository page. **Your code is on the internet** — though not yet running anywhere.

4.5 Why "I deleted it" does not work

Run this demonstration, or at least read it carefully. It is the single most expensive mistake in this workshop.

What happens:

1. You accidentally commit a file with your API key in it, and push
2. You notice, delete the key, commit again, and push
3. The file on GitHub now looks completely clean

And yet anyone who clones your repository can run:

```
git show <earlier-commit>:main.py
```

```
client = anthropic.Anthropic(api_key="sk-ant-api03-REDACTED")
```

The key is still there.

Git's whole purpose is remembering every version. Deleting a line creates a *new* commit; it does not remove the old one. The old version is still in the history, and the history is what gets uploaded.

Rewriting history does not save you either. By the time you notice, the repository has been cloned, cached, and scanned.

> **The only real fix**
>
> If a key has been pushed — even for one minute, even to a private repository — **treat it as stolen.**
> Go to the provider's website, delete that key, and create a new one. It takes thirty seconds and it is the only action that actually works.

Providers also scan public repositories automatically and disable keys they find, often within minutes. Your application then dies with a `401` error at the worst possible time.

This is why `git status` before your first push is worth the three seconds.

4.6 Deploy on Render

You created the account in Part 0. Now we use it.

The steps

1. Go to **https://dashboard.render.com**
2. Click **New** (top right) -> **Blueprint**

   > **Blueprint** is Render's word for "read the `render.yaml` file and set everything up from it". The alternative is filling in a form by hand; the file is better, because it is version-controlled and repeatable.
3. If asked, **connect your GitHub account** and give Render access — either to all repositories, or just to `sensor-triage`
4. Select **`sensor-triage`** from the list
5. Render reads your `render.yaml` and shows what it is about to create. It should show one web service on the free plan
6. Click **Apply** (or **Create**)

Now wait

The first build takes **two to four minutes**. You will see a log scrolling past:

```
==> Cloning from https://github.com/...
==> Running build command 'pip install -r requirements.txt'
    Collecting fastapi==0.141.1
    ...
==> Build successful
```

```
==> Running 'uvicorn main:app --host 0.0.0.0 --port $PORT'
INFO:     Uvicorn running on http://0.0.0.0:10000
==> Your service is live
```

> **Errors and a blank page during this time are normal, not failure.** The service does not exist until the build finishes. Read the log rather than refreshing the address.

See the result

At the top of the page is your address:

```
https://sensor-triage-xxxx.onrender.com
```

Open it. **Open it on your phone.** Send it to someone in another country.

Your application is on the internet.

Check the pieces

Address	Should show
`/`	the page you built
`/health`	`{"status":"ok"}`
`/docs`	the interactive API page

`/docs` being live is worth noticing: anyone can now explore your API. That is often what you want, and occasionally what you do not — real services put authentication in front of it.

4.7 Adding your API key in production

Your deployed application is running with `AI_PROVIDER=true`, which is why it needed no key.

If you have one:

1. In Render, open your service -> **Environment**
2. Find **`ANTHROPIC_API_KEY`** — it exists with no value, because of `sync: false`
3. Paste your key in and **Save**
4. In your own `render.yaml`, change `AI_PROVIDER` to `"false"`
5. Commit and push

That fifth step is not optional, and here is why.

> **The dashboard does not always win**
>
> Changing `AI_PROVIDER` in Render's dashboard **will not work**. `render.yaml` gives it a `value:`, and the file is the source of truth.

> I lost fifteen minutes to this while preparing this workshop: changed the value in the dashboard, watched, and nothing happened.
> **The rule once you use a configuration file:** settings with a `value:` in the file -> change the file and push. settings with `sync: false` -> change them in the dashboard.

4.8 Updating your application

This is the part that makes deployment worth doing properly.

```
git add .
```

```
git commit -m "what you changed"
```

```
git push
```

That is it. Render notices the push and rebuilds automatically. A minute or two later your change is live.

You now have a real development cycle: **edit locally -> test locally -> commit -> push -> it is live.**

4.9 Things that will surprise you

Your app falls asleep

Free services stop after about **15 minutes** with no visitors. The next person to arrive waits **30 to 60 seconds** while it wakes up, staring at a blank tab.

Nothing is broken. It is how free tiers work.

Before any demonstration, open your own link ten minutes early. It costs nothing and it means your visitor gets an awake server.

Your stored readings disappear

Free plans give your service a **temporary disk**. Every deploy gives you a fresh one, and `readings.db` is on it.

Your application still works perfectly. The history is just empty again.

If data must survive, that is the moment to add a managed database — Render offers a free PostgreSQL instance. It is a change to how you connect, not to your application's logic.

The build failed

Open the **Logs** tab and read from the top. The first error is the real one.

Log says	Usually means
`ERROR: Could not find a version...`	a typo or wrong version in `requirements.txt`
`ModuleNotFoundError`	a package you use is missing from `requirements.txt`
`No such file or directory: 'static'`	the `static` folder was not committed
service starts then stops	`startCommand` is wrong, or the port is hardcoded
`yaml: line N`	indentation in `render.yaml` — YAML is strict about spaces

It works locally but not deployed

The usual cause is something that exists on your machine and not on the server:

- a package installed but missing from `requirements.txt`
- a file that `.gitignore` excluded — check with `git status`
- a setting that is in your `.env` but not in Render's Environment tab
- anything pointing at `localhost`, which on a server means **the server**

What you have now

```
your laptop  ──push──>  GitHub  ──reads──>  Render  ──serves──>  anyone
```

- A repository with your project's full history
- An application running on a machine that never sleeps
- A public address that works from any device, anywhere
- HTTPS, a health check, and automatic restarts, none of which you configured
- A one-command update cycle

Part 5 covers where to take it next, and how the other workshops in this series plug into what you have just built.

Part 5 — Where to go next

Your application is finished and deployed. This part is what to add when you need it.

Each section is a pointer with enough code to start, not a full tutorial. Take the ones your project actually needs and ignore the rest.

5.1 Connect a real device

This is the one most of you need, and it requires **no change to your server at all.**

Your `/readings` endpoint accepts JSON. It does not know or care whether that JSON came from a browser, a phone, a Python script, or a microcontroller.

From an ESP32 or Arduino

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>

const char* WIFI_SSID = "your-wifi-name";
const char* WIFI_PASS = "your-wifi-password";
const char* ENDPOINT  = "https://sensor-triage-xxxx.onrender.com/readings";

void setup() {
  Serial.begin(115200);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println(" connected");
}

void loop() {
  float reading = analogRead(34) * 0.1;      // <-- your sensor here

  WiFiClientSecure client;
  client.setInsecure();        // skip certificate checking (fine for a prototype)

  HTTPClient http;
  http.begin(client, ENDPOINT);
  http.addHeader("Content-Type", "application/json");
```

```
  String body = String("{\"sensor\":\"motor_temp\",\"value\":") + reading +
                ",\"unit\":\"C\",\"low\":20,\"high\":60}";

  int status = http.POST(body);
  Serial.printf("sent %.1f -> HTTP %d\n", reading, status);
  http.end();

  delay(60000);                                  // once a minute
}
```

That is the whole integration. Your sensor readings now appear on a web page anyone in the world can open.

From a Python script or a Raspberry Pi

```
import requests

requests.post(
    "https://sensor-triage-xxxx.onrender.com/readings",
    json={"sensor": "motor_temp", "value": 87,
          "unit": "C", "low": 20, "high": 60},
    timeout=10,
)
```

Three practical notes

`client.setInsecure()` skips checking the server's HTTPS certificate. Fine for a prototype on your own service; not what you would ship in a product.

Watch the free tier's sleep. If your device posts once an hour, the service will be asleep every time and each post waits for a wake-up. Posting every few minutes keeps it awake — which is itself a reason to consider a paid plan later.

Send the range from the device or hardcode it in the server. Right now the device sends `low` and `high` every time. For a real deployment you would more likely store safe ranges per sensor in the database, so the device only sends a reading.

5.2 Where the other workshops plug in

Look at the ten sessions in this series. Every other one produces something that lives **on a laptop or on a device**: a trained model, code on a microcontroller, readings in a terminal, a PCB.

Yours is the only one that ends with something anyone can open. That makes this application the natural place for the others to become visible.

Workshop	What it gives you	Where it plugs in
Predictive Analytics & AI Modelling	a trained model	replace `classify()` with your model's prediction, or add an endpoint that runs it
TinyML — Edge AI on Small Devices	a model running on a chip	the device POSTs its prediction to `/readings` as the `value`
Edge AI & Sensor Monitoring	live sensor readings	exactly section 5.1 — no server changes
Designing IoT Systems for Energy Saving	power telemetry	same endpoint; the "reading" is watts
Software Algorithms & Control	control logic on a device	POST the state, display it, and read decisions back
Engineering AI Prompting	better prompts	the `SYSTEM` and `prompt` strings in `ai.py`

Replacing `classify()` with a trained model

If Workshop 3 gave you a model, the change is small:

```
import joblib

model = joblib.load("model.pkl")


def classify(value: float, low: float, high: float) -> str:
    prediction = model.predict([[value, low, high]])[0]
    return prediction        # "normal" | "warning" | "critical"
```

Everything else — the endpoint, the database, the page, the deployment — is unchanged. **That is what a clean boundary buys you.**

> One caution: add `scikit-learn` (or whatever you trained with) to `requirements.txt`, and commit the model file. Model files can be large; anything over about 100 MB will not fit in a normal git repository.

5.3 Keep your data when you redeploy

Free hosting wipes the disk on every deploy, so `readings.db` disappears.

Fix: use a managed database instead of a file.

1. In Render: **New -> PostgreSQL**, choose the free plan
2. Copy the **Internal Database URL** it gives you
3. Add it as an environment variable, `DATABASE_URL`
4. Install `psycopg[binary]` and change how you connect

The concepts you learned do not change — a table, an `INSERT`, a `SELECT`. Only the connection does.

> Render's free PostgreSQL expires after a period and then needs recreating. Check the current terms before relying on it for anything that matters.

5.4 Make it feel fast

Your `/readings` endpoint waits for the AI before replying. With a fast model that is fine. With a slow one it is not — measured on the same question, five times in a row: **4.7s, 4.9s, 7.5s, 17.1s, 26.4s.**

The problem is not that it is slow. It is that you **cannot predict** how slow, so you cannot promise the user anything.

The fix: answer first, explain afterwards

1. Compute the severity and save the row with `ai_status = "pending"`
2. **Return immediately** — the visitor has their answer in milliseconds
3. Do the AI call in a background task, and update the row when it finishes
4. The page polls until the explanation appears

FastAPI has this built in:

```
from fastapi import BackgroundTasks


def fill_in_explanation(reading_id: int, ...):
    """Runs AFTER the response has already been sent."""
    advice = ai.explain(...)
    with db() as conn:
        conn.execute(
            "UPDATE readings SET reason=?, action=?, ai_status='done' WHERE id=?",
            (advice.reason, advice.action, reading_id),
        )


@app.post("/readings")
def create_reading(reading: Reading, background: BackgroundTasks):
    severity = classify(reading.value, reading.low, reading.high)
    # ... insert with ai_status='pending', get new_id ...

    background.add_task(fill_in_explanation, new_id, ...)   # queued, not awaited

    return {"id": new_id, "severity": severity, "ai_status": "pending"}
```

Measured on this application: the endpoint returned in **~30 ms** instead of waiting 9 to 50 seconds. The slow part is still slow; it just stopped being the visitor's problem.

> You have seen this pattern before. A messaging app shows your message instantly with one tick, then updates the delivery status a moment later. It did not wait for the network before showing you your own message.
> This applies far beyond AI. Any slow third party — payments, email, image processing — belongs behind this same structure.

5.5 Run the AI free, on your own machine

You can run a small language model locally with **Ollama**. No account, no key, no cost, and it works with no internet connection.

```
ollama pull llama3.2:3b
```

Then add a branch to `explain()` in `ai.py` that posts to `http://localhost:11434/api/chat`.

The trap that will cost you a day

`localhost` means "the computer running this code".

On your laptop, that is your laptop, and Ollama is there. On your deployed server, that is the server — and Ollama is not there. Your AI silently stops working and the application keeps running as though nothing is wrong.

You cannot fix it by installing Ollama on the server either. A small model needs around **4 GB** of memory; free hosting gives you about **512 MB**. It is not a configuration problem you can solve; it does not fit.

So decide your target before you build

Your project runs…	Use
on a laptop at a table, possibly with no wifi	a local model — free, private, offline
at a public address people open remotely	a hosted API — needs a key, costs cents

Both are valid. Choosing late is what costs you.

5.6 Control what you spend

Two limits, and you want both.

1. On your provider account. Set a hard monthly cap in their console before you write any code. This is the one that actually protects you.

2. In your application. Refuse to send the request once you have spent enough today:

```
def check_budget():
    """Call BEFORE spending money. Raise rather than charge."""
    spent = todays_spend()          # you store this
    if spent >= MAX_USD_PER_DAY:
        raise RuntimeError(f"Daily cap reached (${spent:.4f})")
```

Then record the real cost afterwards, using the token counts the API reports back rather than an estimate:

```
usage = response.usage
cost = (usage.input_tokens / 1e6) * PRICE_IN \
     + (usage.output_tokens / 1e6) * PRICE_OUT
```

Normal use costs almost nothing. What costs money is a bug that retries forever overnight, or a key someone else is using. The cap catches both.

5.7 Know when the AI has failed

A subtle one, and worth understanding.

Suppose you make `explain()` fall back to a plain sentence when the AI is unreachable. That is good design — the application keeps working.

But it creates a new problem: **the failure becomes invisible.** The app looks fine, the output looks plausible, and the AI has not run for three days.

Two rules:

Log every fallback, with the reason.

```
except Exception as e:
    logging.warning("AI unavailable (%s), using fallback: %s", type(e).__name__, e)
    return fallback
```

Never let the interface claim a model wrote something a model did not write. Store where the text came from — the model name, or `"rule"`, or `"fallback:ConnectionError"` — and show it.

> This happened while building this workshop. The dashboard said “from the AI model” above text an `if` statement had written. It was only noticed because someone asked what the label meant.
> **Degrade gracefully, but log loudly.** Any fallback path in any system needs to be visible, or you will not find the bug.

5.8 Where to learn more

Topic	Where
FastAPI	**fastapi.tiangolo.com** — genuinely excellent, start with the tutorial
Git	**git-scm.com/book** — free, thorough
HTTP and the web	**developer.mozilla.org** (MDN) — the reference everyone uses
Render	**render.com/docs**
Databases	the PostgreSQL tutorial, once SQLite stops being enough

The three things to learn next

1. **Authentication** — how to require a login. Right now anyone can post to your endpoint. FastAPI's security documentation covers this.
2. **Testing** — writing code that checks your code. `pytest` plus FastAPI's `TestClient`. Your `classify()` function is an ideal first test.
3. **Managed databases** — section 5.3, when data needs to survive.

What you actually learned

The application is a vehicle. What transfers is underneath it:

- **A backend receives requests and decides what to send back.** Every web service works this way.
- **The frontend is public and the backend is not.** That single fact decides where secrets live.
- **Validate input at the edge**, because anyone can send anything.
- **Know what each component is good at.** Arithmetic to code, language to a language model.
- **Anything slow or external is allowed to fail** — but make the failure visible.
- **Pin your dependencies**, or your build breaks on a day you did not touch it.
- **Config belongs outside your code**, so the same program runs anywhere.

None of that is FastAPI, Render, or this application. It is how software gets built, and the next thing you build will have the same shape.

Appendix A — Glossary

Terms in the order you are likely to meet them.

Terminal — a window where you type commands instead of clicking.

Virtual environment (`.venv`) — a private copy of Python for one project, so its packages do not affect anything else on your machine.

Server — a computer that stays switched on, waiting for requests. Nothing more special than that.

Request / Response — a question sent over the internet, and the answer. Every action on the web is this pair.

GET / POST — GET asks for something; POST sends something.

Status code — the number in a response saying how it went. 200 fine, 404 nothing at that address, 422 wrong shape, 500 your code crashed.

Endpoint — one address your server answers, and the code behind it.

Port — the number after the colon in `127.0.0.1:8000`. One computer runs many programs; the port says which one you want.

localhost / 127.0.0.1 — "this computer, right here". What "this computer" means depends on where the code is running.

Frontend — the page in the visitor's browser. Public: anyone can read all of it.

Backend — your program, running on a server. Private.

Full-stack — building the frontend, the backend, and the data storage.

JSON — the text format programs use to exchange data. Looks like a Python dictionary.

API — a way for your program to ask another program a question over the internet and get an answer.

API key — a secret string identifying your account with a service. Behaves like a bank card with no PIN.

Environment variable — a setting supplied to a program when it runs, rather than written inside it. Where secrets belong.

SQLite — a complete database inside a single file. Nothing to install.

Parameterised query — passing SQL and its values separately, using `?`. What prevents SQL injection.

The cloud — someone else's computer, in a building somewhere, rented to you.

Static hosting — a host that serves finished files and runs no code of yours.

Platform hosting — you give them code and a start command; they run it. What we use.

Server hosting — you rent a bare machine and administer it yourself.

Serverless — you give them one function; it runs on demand and costs nothing when idle.

Container / Docker — packaging code together with everything it needs, so it runs identically everywhere.

HTTPS — encrypted traffic between browser and server. The padlock.

DNS — the system that turns a name like `example.com` into a number.

Git — the program that records versions of your code, on your machine.

GitHub — a website that stores copies of git repositories online.

Repository (repo) — one project's folder, with all of its history.

Commit — a saved checkpoint in that history.

Remote / Push — a copy of your repository elsewhere, and sending your commits to it.

Deploy — moving your application onto a server that stays on, at a public address.

Cold start — the delay when a sleeping free service has to wake up.

Pinning — writing exact version numbers in `requirements.txt`, so future builds install the same thing.

Appendix B — Command reference

Terminal basics

Command	What it does
`cd foldername`	go into a folder
`cd ..`	go up one folder
`dir`	list this folder's contents (macOS/Linux: `ls`)
`cls`	clear the screen (macOS/Linux: `clear`)
`Ctrl + C`	stop whatever is running
`Tab`	complete a name you have started typing

Python and the project

Command	What it does
`python --version`	check Python is installed
`python -m venv .venv`	create the virtual environment
`.venv\Scripts\activate`	turn it on (Windows)
`source .venv/bin/activate`	turn it on (macOS/Linux)
`pip install <package>`	install a package
`pip freeze`	list exact installed versions
`uvicorn main:app --reload`	run the server

Git

Command	What it does
`git init`	start tracking this folder
`git add .`	stage all changes
`git status`	**check what you are about to upload**
`git commit -m "message"`	save a checkpoint
`git remote add origin <url>`	connect to GitHub
`git push`	send commits to GitHub
`git log --oneline`	list your checkpoints

Useful addresses while developing

Address	What it shows
`127.0.0.1:8000`	your web page
`127.0.0.1:8000/health`	the health check
`127.0.0.1:8000/docs`	the interactive API page
`127.0.0.1:8000/readings`	the stored readings, as JSON

Appendix C — Troubleshooting

Setup

What you see	What it means	Fix
`python is not recognized`	the PATH box was missed on install	reinstall Python, tick **Add python.exe to PATH**
`git is not recognized`	terminal opened before installing	close it, open a new one
`running scripts is disabled`	Windows blocks scripts by default	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`cd` seems to do nothing	wrong drive, in Command Prompt	type `E:` on its own first
`The system cannot find the path`	a space in the path	wrap it in quotes: `cd "My Folder"`
macOS: `python: command not found`	macOS uses `python3`	use `python3` and `pip3`

Building

What you see	What it means	Fix
`ModuleNotFoundError`	package missing, or venv not active	activate the venv, then `pip install`
`Address already in use`	an old server is still running	stop it, or `--reload --port 8001`
`/health` returns your HTML page	`app.mount("/")` is above your endpoints	move it to the last line of the file
`no column named reason`	the table was created at an earlier stage	add the `ALTER TABLE` migration, or delete `readings.db`
Your `.env` is called `.env.txt`	Windows hides file extensions	turn extensions on, rename it
`422 Unprocessable Entity`	your JSON does not match the `Reading` class	check field names and types
The page loads but Send does nothing	look at the browser console (`F12`)	the error is usually named there

Deploying

What you see	What it means	Fix
Build fails: `Could not find a version`	wrong version in `requirements.txt`	check against `pip freeze`
Build fails: `yaml: line N`	indentation in `render.yaml`	YAML is strict; use spaces, not tabs
Service starts then stops	port hardcoded, or wrong start command	use `--port $PORT`
Address shows nothing, logs look fine	not listening on `0.0.0.0`	add `--host 0.0.0.0`
First visit takes 30-60 seconds	free service was asleep	normal; open your link before a demo
Stored readings gone after deploying	free plans wipe the disk	use a managed database if it matters
Works locally, not deployed	something exists on your machine only	check `requirements.txt`, `git status`, and environment settings

Reading an error

Three habits worth more than any table above:

1. **Read the first error, not the last.** Later errors are usually consequences.
2. **Read the whole message.** It almost always names the file, the line, and the problem.
3. **Check where you are** before assuming the code is wrong. Most “it does not work” moments are “I am in the wrong folder” or “the virtual environment is not on”.

Appendix D — Checklist

Print this, or keep it open.

Before you start

- ☐ `python --version` shows 3.10 or higher
- ☐ `git --version` works
- ☐ VS Code installed
- ☐ File name extensions visible (Windows)
- ☐ Logged in at github.com
- ☐ Logged in at render.com

While building

- ☐ Virtual environment activated — you can see `(.venv)`
- ☐ `uvicorn main:app --reload` running, left running
- ☐ `app.mount("/")` is the **last** line of `main.py`
- ☐ `.gitignore` created **before** `.env`
- ☐ `readings.db` deleted after adding table columns

Before you push

- ☐ `git status` run, and read
- ☐ `.env` is **not** in the list
- ☐ `readings.db`, `.venv`, `__pycache__` are **not** in the list
- ☐ No key written in `render.yaml`

After deploying

- ☐ `/health` returns `{"status":"ok"}`
- ☐ `/` shows your page
- ☐ `/docs` loads
- ☐ Opened it on a phone, on mobile data, not on your own wifi

Colophon

Written for **Project Nexus 2026 — AI Innovation Challenge**, organised by the IEEE Multimedia University Student Branch, with co-organisers E3S2 UTP, IEEE PES MMU SBC, EWB MMU, and IEM MMU.

Every command and every line of code in this guide was executed and verified before publication. The measured figures quoted in Part 3 and Part 5 — model accuracy, response times, concurrency behaviour — were produced by running the tests described, not taken from other sources.

The companion repository contains all six stage folders, each a complete working application.